Kie.ai API Documentation - Nano Banana Pro

For Bayt Alebaa Image Generation Platform

📌 Overview

This documentation covers the integration of **Kie.ai's Nano Banana Pro API** for generating interior design images with building materials (tiles, ceramics, parquet, etc.)

🔗 Important Links

- **API Documentation**: https://docs.kie.ai/
- **Nano Banana Pro Docs**: https://docs.kie.ai/market/google/pro-image-to-image
- **Get API Key**: https://kie.ai/api-key
- **Pricing**: https://kie.ai/pricing
- **Playground (Test)**: https://kie.ai/nano-banana-pro

💰 Pricing

Resolution	Price per Image
1K - 2K	$0.09
4K	$0.12

🔐 Authentication

Every request must include these headers:

```
Authorization: Bearer YOUR_API_KEY
Content-Type: application/json
```

Get your API key from: https://kie.ai/api-key

🚀 API Endpoints

1. Create Image Generation Task

Endpoint: `POST https://api.kie.ai/api/v1/jobs/createTask`

Request Body:

```json
{
    "model": "nano-banana-pro",
    "callBackUrl": "https://your-domain.com/api/callback",
    "input": {
        "prompt": "Your detailed prompt here",
        "image_input": ["https://url-to-reference-image.jpg"],
        "aspect_ratio": "16:9",
        "resolution": "2K",
        "output_format": "png"
    }
}
```

Response:

```json
{
    "code": 200,
    "msg": "success",
    "data": {
        "taskId": "task_nano-banana-pro_1765178625768"
    }
}
```

---

2. Get Task Status/Result

Endpoint: `GET https://api.kie.ai/api/v1/jobs/recordInfo?taskId=YOUR_TASK_ID`

Headers:

```
Authorization: Bearer YOUR_API_KEY
```

Response:

```json
{
    "code": 200,
    "msg": "success",
    "data": {
        "taskId": "task_12345678",
        "model": "nano-banana-pro",
        "state": "success",
        "resultJson": "{\"resultUrls\":[\"https://example.com/generated-image.j
        "costTime": 15000,
        "completeTime": 1698765432000,
        "createTime": 1698765400000
    }
}
```

Task States:

- `pending` - Task is queued
- `processing` - Task is being processed
- `success` - Task completed successfully
- `failed` - Task failed

---

3. Check Account Credits

Endpoint: `GET https://api.kie.ai/api/v1/chat/credit`

Headers:

```
Authorization: Bearer YOUR_API_KEY
```

📝 Input Parameters

Parameter	Type	Required	Description
`model`	string	Yes	Always use `"nano-banana-pro"`

Parameter	Type	Required	Description
`callBackUrl`	string	No	Webhook URL to receive results
`input.prompt`	string	Yes	Text description of the image
`input.image_input`	array	No	Array of reference image URLs (up to 8)
`input.aspect_ratio`	string	No	Image ratio (see options below)
`input.resolution`	string	No	`"1K"`, `"2K"`, or `"4K"`
`input.output_format`	string	No	`"png"` or `"jpg"`

Aspect Ratio Options:

- `1:1` - Square (social media)
- `2:3` - Portrait
- `3:2` - Landscape
- `3:4` - Portrait
- `4:3` - Classic
- `4:5` - Portrait (Instagram)
- `5:4` - Landscape
- `9:16` - Mobile/Stories
- `16:9` - Widescreen
- `21:9` - Ultra-wide
- `auto` - Automatic

Code Examples

JavaScript/Node.js

```javascript
```

```javascript
const KIE_API_KEY = 'your_api_key_here';
const KIE_BASE_URL = 'https://api.kie.ai';

// Step 1: Create generation task
async function generateImage(prompt, referenceImageUrl, options = {}) {
    const response = await fetch(`${KIE_BASE_URL}/api/v1/jobs/createTask`, {
        method: 'POST',
        headers: {
            'Authorization': `Bearer ${KIE_API_KEY}`,
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({
            model: 'nano-banana-pro',
            callBackUrl: options.callbackUrl || null,
            input: {
                prompt: prompt,
                image_input: referenceImageUrl ? [referenceImageUrl] : [],
                aspect_ratio: options.aspectRatio || '16:9',
                resolution: options.resolution || '2K',
                output_format: options.format || 'png'
            }
        })
    });

    const data = await response.json();
    return data.data.taskId;
}

// Step 2: Poll for results
async function getTaskResult(taskId) {
    const response = await fetch(
        `${KIE_BASE_URL}/api/v1/jobs/recordInfo?taskId=${taskId}`,
        {
            headers: {
                'Authorization': `Bearer ${KIE_API_KEY}`
            }
        }
    );

    const data = await response.json();
    return data.data;
}
```

```
// Step 3: Wait for completion
async function waitForResult(taskId, maxAttempts = 60) {
    for (let i = 0; i < maxAttempts; i++) {
        const result = await getTaskResult(taskId);

        if (result.state === 'success') {
            const resultData = JSON.parse(result.resultJson);
            return resultData.resultUrls[0];
        }

        if (result.state === 'failed') {
            throw new Error(`Task failed: ${result.failMsg}`);
        }

        // Wait 3 seconds before next poll
        await new Promise(resolve => setTimeout(resolve, 3000));
    }

    throw new Error('Task timeout');
}

// Complete usage example
async function generateInteriorImage(productImageUrl, placement, settings) {
    const prompt = buildPrompt(placement, settings);

    const taskId = await generateImage(prompt, productImageUrl, {
        aspectRatio: settings.aspectRatio,
        resolution: settings.resolution
    });

    const imageUrl = await waitForResult(taskId);
    return imageUrl;
}
```

Python

```python
```

```python
import requests
import time
import json

KIE_API_KEY = 'your_api_key_here'
KIE_BASE_URL = 'https://api.kie.ai'

headers = {
    'Authorization': f'Bearer {KIE_API_KEY}',
    'Content-Type': 'application/json'
}

def generate_image(prompt, reference_image_url=None, options=None):
    """Create an image generation task"""
    if options is None:
        options = {}

    payload = {
        'model': 'nano-banana-pro',
        'callBackUrl': options.get('callback_url'),
        'input': {
            'prompt': prompt,
            'image_input': [reference_image_url] if reference_image_url else []
            'aspect_ratio': options.get('aspect_ratio', '16:9'),
            'resolution': options.get('resolution', '2K'),
            'output_format': options.get('format', 'png')
        }
    }

    response = requests.post(
        f'{KIE_BASE_URL}/api/v1/jobs/createTask',
        headers=headers,
        json=payload
    )

    data = response.json()
    return data['data']['taskId']

def get_task_result(task_id):
    """Get task status and result"""
    response = requests.get(
        f'{KIE_BASE_URL}/api/v1/jobs/recordInfo',
        headers=headers,
```

```
        params={'taskId': task_id}
    )

    return response.json()['data']

def wait_for_result(task_id, max_attempts=60, interval=3):
    """Poll until task completes"""
    for _ in range(max_attempts):
        result = get_task_result(task_id)

        if result['state'] == 'success':
            result_data = json.loads(result['resultJson'])
            return result_data['resultUrls'][0]

        if result['state'] == 'failed':
            raise Exception(f"Task failed: {result.get('failMsg', 'Unknown erro

        time.sleep(interval)

    raise Exception('Task timeout')

# Usage example for Bayt Alebaa
def generate_interior_with_product(product_image_url, prompt, settings):
    """
    Generate interior image with product material

    Args:
        product_image_url: URL of the tile/ceramic/parquet image
        prompt: Description of the space and how to use the product
        settings: Dict with aspect_ratio, resolution, etc.
    """
    task_id = generate_image(
        prompt=prompt,
        reference_image_url=product_image_url,
        options=settings
    )

    image_url = wait_for_result(task_id)
    return image_url
```

🏠 Prompt Template for Bayt Alebaa

Use this template for generating interior images with building materials:

```
{quality} interior design photography of {space_type},
featuring {product_description} as the {placement},
{design_style}, {lighting}, {camera_angle}, {lens_type},
{material_focus}, {composition}, {mood},
photorealistic, architectural visualization, professional interior
photography
```

Example Prompt:

```
ultra high quality, 4K resolution, highly detailed,
interior design photography of spacious living room,
featuring gray marble floor tiles with subtle veining as the floor covering,
luxury high-end design with premium materials and sophisticated details,
natural daylight streaming through large windows creating bright and airy
atmosphere,
corner composition showing two walls and floor depth,
shot with 24mm wide angle lens for expansive architectural view,
emphasis on material texture with visible surface details,
elegant refined atmosphere with professional commercial quality,
photorealistic, architectural visualization, professional interior
photography
```

⚠️ Important Notes

1. **Asynchronous API**: All tasks are asynchronous. You submit a task and poll for results.
2. **Rate Limits**:
   - 20 requests per 10 seconds
   - ~100+ concurrent tasks allowed
3. **Data Retention**:
   - Generated images: stored for 14 days
   - Download immediately and save to your storage
4. **Download Links**: Valid for only 20 minutes
5. **Callback Option**: For production, use `callBackUrl` instead of polling

---

🔄 Callback Webhook

If you provide a `callBackUrl`, Kie.ai will POST the result to your endpoint:

```json
{
    "taskId": "task_nano-banana-pro_1765178625768",
    "state": "success",
    "resultJson": "{\"resultUrls\":[\"https://...\"]}"
}
```

Your endpoint should return HTTP 200 to acknowledge receipt.

🛠️ Error Handling

Code	Message	Solution
401	No access permissions	Check API key
422	Validation error	Check input parameters
429	Too many requests	Reduce request rate
500	Server error	Retry after delay

📞 Support

- Email: support@kie.ai
- Discord: Available from dashboard
- Telegram: Available from dashboard
- Support Hours: UTC 21:00 – UTC 17:00 (next day)

✅ Quick Checklist for Integration

- [ ] Get API key from https://kie.ai/api-key

- [ ] Add credits to account
- [ ] Test in Playground first: https://kie.ai/nano-banana-pro
- [ ] Implement task creation endpoint
- [ ] Implement polling or callback handler
- [ ] Handle errors gracefully
- [ ] Save generated images to your own storage
- [ ] Monitor credit usage